

محاسبي — الخطة التقنية (Tech Plan)

الوثيقة دي هي دليل التنفيذ. الـ Features والـ Billing موجودين في `PLAN.md`، وهنا **إزاي هبنينهم**: الـ Stack، الهيكل، القواعد، الداتايز، الـ APIs، المزامنة الأوفلاين، الـ Real time، الاختبارات، النشر، وخطة أول أسابيع يوم بيوم.

0. القرارات الأساسية (ملخص)

الموضوع	القرار
Backend	Laravel (آخر إصدار مستقر x/13.x.12) + Modular Monolith: Modules — PHP 8.4 مستقلة تتفعل لكل محل حسب اشتراكه
التواصل بين الـ Modules	Event-Driven: Domain Events + Transactional Outbox + Listeners في الـ Queue (مش Event Sourcing)
قاعدة البيانات	PostgreSQL 17
Real time	Laravel Reverb (WebSockets) + Laravel Echo
Queues	Redis + Laravel Horizon
Auth	Sanctum : كوكيز للوحة الويب، Tokens لأجهزة الـ POS وتطبيق المالك
الصلاحيات	spatie/laravel-permission بوضع Teams (الـ Team = المحل)
Frontend	Nuxt 4 + TypeScript + Nuxt UI (RTL) + Pinia — تطبيق واحد فيه اللوحة والكاشير
الكاشير أوفلاين	PWA + IndexedDB (Dexie) + Outbox Sync
تطبيق المالك	Flutter + Riverpod + Dio + go_router
لوحة إدارة المنصة (إحنا)	Filament داخل نفس مشروع Laravel
توثيق الـ API	Scramble (OpenAPI تلقائي) ← توليد TypeScript types للـ Nuxt
الفلوس	integer بالقرش + brick/money
IDs	UUIDv7 (بيتولد على الجهاز في الأوفلاين ومترتب زمنياً)
الاختبارات	Pest + Larastan + Pint / Vitest + Playwright
النشر	Cloudflare (DNS/R2) + Sentry + (Laravel Forge أو Coolify) Docker + VPS

ليه Modular Monolith؟ مشروع واحد، Deploy واحد، داتايز واحدة — أسرع حاجة لمطور واحد. والتقسيم لـ Modules بحدود واضحة، والتواصل بينها بالـ Events، بيخلي أي Module يتفعل أو يتفعل لمحل معين من غير ما يآثر على الباقي، ويسهل فصل أي جزء كخدمة لوحده بعدين لو احتجنا.

1. هيكل الـ Repo (Monorepo)

```
muhasabi-egypt/
├── apps/
│   ├── api/           # Laravel - API + Reverb + Horizon + Filament (المنصة admin)
│   ├── web/           # Nuxt 4 - لوحة المحل + الكاشير (PWA)
│   └── owner/         # تطبيق المالك - Flutter
├── packages/
│   └── api-types/     # TypeScript types من OpenAPI (Scramble)
├── infra/
│   ├── docker/       # Dockerfiles + nginx + supervisor
│   └── compose.yml    # بيئة التطوير المحلية
├── docs/              # ADRs + ملاحظات تقنية
├── PLAN.md
├── TECH_PLAN.md
└── .github/workflows/ # CI
```

2. بيئة التطوير المحلية

فيه: `infra/compose.yml`

Port	الاستخدام	Service
5432	الداتابيز	pgsql (postgres:17)
6379	Queue + Cache + Sessions	redis (redis:7)
8025	استقبال الإيميلات في التطوير	mailpit
9000	S3 محلي (صور البطايق، الأجهزة، المرفقات)	minio
8000	FrankenPHP أو php artisan serve	api
8080	php artisan reverb:start	reverb
—	php artisan horizon	horizon
3000	pnpm dev (Nuxt)	web

أو ببساطة **Laravel Sail** للـ API، والـ Nuxt شغال على الجهاز مباشرة.

3. هيكل الـ Backend (Laravel)

3.1 التقسيم لـ Modules

```
apps/api/app/
├── Modules/
│   ├── — (للمحل Module دائماً شغالة، مش بتظهر كـ Platform) —
│   ├── Identity/      # Tenants, Branches, Users, Roles, Devices, PIN login
│   └── ModuleManager/ # تفعيلها لكل محل + Modules كتالوج الـ
```

— Billing/	# Plans, Subscriptions, Invoices, Payments, Gateways
— Messaging/	# MessageTemplates, MessageLogs, wa.me links
— Sync/	# Push/Pull للأجهزة الأوفلاين
— Core (الاشتراكات) —	
— Catalog/	# Categories, Brands, DeviceModels, Products, Variants,
Compatibility	
— Inventory/	# StockLevels, StockMovements, StockLots, SerialItems,
Stocktake	
— Sales/	# Sales, SaleItems, Payments, Returns, Holds (POS)
— Customers/	# Customers, CustomerLedger (الآجل)
— Suppliers/	# Suppliers, Purchases, SupplierLedger
— Cash/	# Shifts, CashMovements, Expenses
— Reports/	# Read models + Query classes
— Optional (بالتفعل حسب الاشتراك) —	
— Repairs/	# الصيانة
— Imports/	# الاستيراد
— SupplierReturns/	# مرجعات الموردين حسب المصدر
— UsedDevices/	# المستعمل
— Services/	# كروت الشحن + تحويلات المحافظ
— Installments/	# التقسيط
— MultiBranch/	# التحويلات بين الفروع + التقارير المجمعة
— EInvoicing/	# ETA
— Support/	# Money, Tenancy, Phone, Numbering, Events (Outbox) ...

جوه كل Module:

```

Modules/Repairs/
— module.php          # Manifest: key, نوع, اسم, dependencies, إعدادات, قائمة, صلاحيات,
— RepairsServiceProvider.php # routes/listeners/policies/migrations
— Contracts/          # تانية مسموح تستخدمها Modules اللي (Interfaces) الواجهة العامة ←
— Events/             # بيطلعها Module الأحداث العامة اللي ال ← (برضه Public API)
— Listeners/          # تانية Modules ردود أفعاله على أحداث
— Models/ Enums/ Actions/ Data/ Policies/
— Http/ Controllers/ Requests/ Resources/
— Database/ migrations/ seeders/ factories/
— routes.php
— Tests/

```

الحدود: أي Module مايلمسش Models أو جداول Module تاني. المسموح بس: Contracts/ و Events/ بتاعته (وده بيتفرض بـ Pest Arch tests).

3.2 قواعد الكود (إجبارية)

- **Controllers رفيعة:** Validation في API Resource ← Action ← Form Request. مفيش Business logic في ال Controller.
- **Action لكل عملية** (handle() واحدة), و Service لما عمليات كثير تشارك نفس ال state.
- كل حاجة Enums: **typed** بدل ال return types, properties, readonly, strings.
- **كل عملية بتلمس فلوس أو مخزون جوه DB::transaction()**, وال Jobs/Events بتتبع بعد ال commit (afterCommit).
- مفيش Model::all() ولا get() من غير حد — Pagination دايمًا.

- مفيش `env()` برا ملفات `config/`.
- Policies لكل Model — مفيش `if ($user->role == ...)`.
- Migrations قابلة للرجوع (`down()`).
- Pint + Larastan (Level 8) لازم يعدّوا قبل أي `merge`.

3.3 الحزم (Packages)

```
composer require laravel/sanctum laravel/reverb laravel/horizon laravel/pennant \
    spatie/laravel-permission spatie/laravel-query-builder spatie/laravel-activitylog \
    spatie/laravel-medialibrary brick/money propaganistas/laravel-phone \
    dedoc/scramble filament/filament barryvdh/laravel-dompdf # أو spatie/laravel-pdf
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan \
    laravel/pint laravel/telescope
```

الحزمة	ليه
<code>spatie/laravel-permission</code>	أدوار وصلاحيات لكل محل (Teams)
<code>spatie/laravel-query-builder</code>	فلتر و ترتيب في الـ APIs بشكل موحد
<code>spatie/laravel-activitylog</code>	الـ Audit Log
<code>spatie/laravel-medialibrary</code>	الصور والمرفقات على S3/R2
<code>brick/money</code>	حسابات الفلوس من غير أخطاء float
<code>propaganistas/laravel-phone</code>	توحيد أرقام الموبايل المصرية (wa.me)
<code>laravel/pennant</code>	Feature flags حسب الباقية
<code>dedoc/scramble</code>	OpenAPI تلقائي من الكود
<code>filament/filament</code>	لوحة إدارة المنصة (المحلات، الاشتراكات، الموافقات اليدوية)

3.4 نظام الـ Modules (تفعيل لكل محل)

الـ Manifest

```
// Modules/Imports/module.php
return new ModuleManifest(
    key: 'imports',
    name: 'الاستيراد',
    tier: ModuleTier::Optional, // Platform | Core | Optional
    dependsOn: ['inventory', 'suppliers'],
    permissions: ['imports.view', 'imports.manage', 'imports.payments'],
    menu: [new MenuItem('imports.index', 'الاستيراد', icon: 'i-lucide-ship', permission:
'imports.view')],
    settings: ['default_cost_allocation' => 'value'],
    seeders: [DefaultImportCostTypesSeeder::class],
);
```

الحالة	المعنى	إيه اللي بيظهر
not_entitled	مش في اشتراكه	مش في القائمة، بيظهر بس في صفحة "ال Modules" بزوار "فعل"
trial	تجربة مؤقتة (مثلاً 14 يوم)	شغال بالكامل + بانر المدة الباقية
enabled	في الاشتراك ومتفعل	شغال بالكامل
disabled	في الاشتراك بس المالك قفله	مخفي، والداتا محفوظة
read_only	الاشتراك خلص أو اتلغى	يقدر يشوف ويصدّر الداتا القديمة بس

- **ال Entitlement (من ال Billing) منفصل عن التفعيل (قرار المالك):** الاشتراك بيقول "مسموح له"، والمالك يقرر "عايز يشوفه ولا لأ".
- **ال Dependencies:** تفعيل Module بيتأكد إن اللي بيعتمد عليه متفعل. وقفل Module مش مسموح لو في Module متفعل معتمد عليه.
- **أول تفعيل:** بيشغل ال Seeders بتاعته (مثلاً قوايم الأعطال للصيانة، والقوالب) ويطلع ModuleEnabled.
- **الإلغاء عمره ما يمسح داتا.** ال Module بيبقى read_only، ولو رجع يتفعل كل حاجة ترجع زي ما هي.
- **عند التسجيل:** المالك يختار نوع المحل (إكسسوارات / صيانة / إكسسوارات + صيانة / مستورد / جملة) ← نقترح Modules وباقية مناسبة.

التطبيق في الكود

```
// Routes
Route::middleware(['auth:sanctum', 'tenant', 'module:imports'])->prefix('imports')->group(...);

// في أي مكان
if (Modules::enabled('repairs')) { ... }

// ModuleGate: Cache محل لكل ModuleEnabled / ModuleDisabled / SubscriptionChanged
final class ModuleGate
{
    public function enabled(string $key, ?Tenant $tenant = null): bool;
    public function state(string $key, ?Tenant $tenant = null): ModuleState;
    /** @return list<string> */
    public function enabledKeys(?Tenant $tenant = null): array;
}
```

- 403 ← module:{key} Middleware module_not_enabled لو مش متفعل (وال Frontend يعرض صفحة "فعل ال Module").
- الصلاحيات: أدوار المحل بتشوف صلاحيات ال Modules المتفعله بس.
- ال Listeners بتاعة Module مقفول **هابتشغلش** للمحل ده (شوف 3.5).

- `GET /api/v1/me` يرجع `modules: [{key, state, trial_ends_at}]` + القائمة الجاهزة — الـ Nuxt والـ Flutter والـ POS الأوفلاين يبينوا القائمة والـ Routes منها.
- الـ Sync pull بيعت داتا الـ Modules المتفّعة بس.

Event-Driven Design 3.5

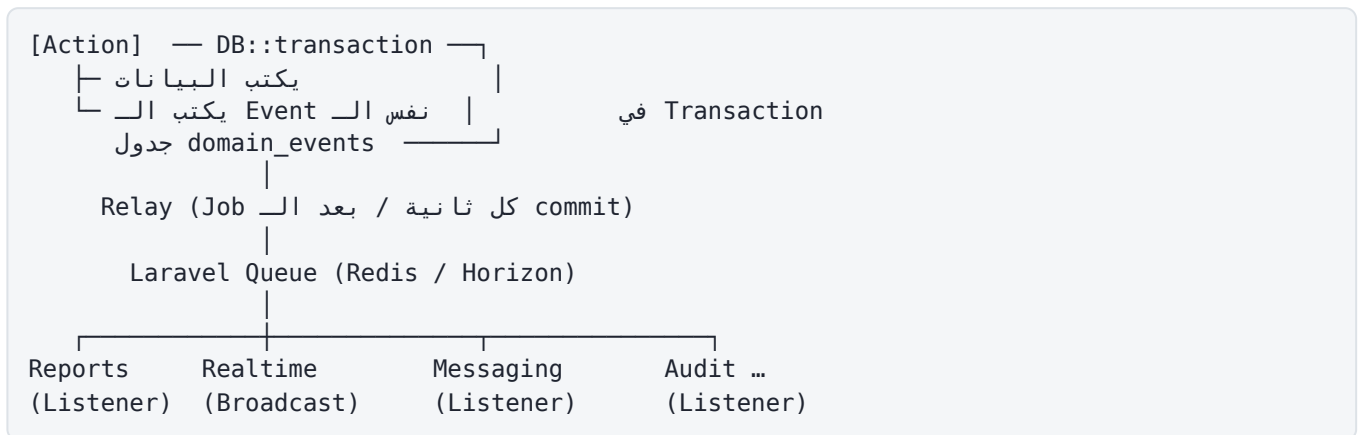
الفكرة: كل Module يعمل شغله ويعلن "حصل إيه" كـ **Domain Event**. الـ Modules الثانية بتسمع وتتصرف. مفيش Module بيكلم الثاني عشان "يعمله حاجة" إلا في حالة واحدة (تحت).

القاعدة الذهبية: Event ولا Contract؟

السؤال	الطريقة	مثال
لو رد الفعل فشل، لازم العملية الأصلية كلها تفشل؟	Contract (استدعاء مباشر لـ Interface جوه نفس الـ Transaction)	البيع لازم يخضم المخزون، والبيع الآجل لازم يتسجل على حساب العميل
ممکن يتأخر ثواني أو يتعاد من غير مشكلة؟	Event (Async في الـ Queue)	التقارير، الـ Real time، الإشعارات، تنبيه النواقص، الـ Audit، عدّادات الاستخدام، اقتراح رسالة WhatsApp

كده بنكسب الاتنين: **اتساق** الفلوس والمخزون، و**فصل** كامل لكل الباقي.

Transactional Outbox (عشان ولا Event يضيع)



- الـ Event بيتسجل في `domain_events` مع البيانات. لو الـ Transaction فشلت، الـ Event مش موجود. لو نجحت، مضمون يتبع حتى لو السيرفر وقع.
- الـ Relay يقرأ الأحداث اللي لسه ماتنشرتش (`published_at IS NULL`) بالترتيب وبيعته لـ `Listeners`، ويعلمها.
- **كل Listener Idempotent**: جدول `Unique processed_events (event_id, listener)` — لو الحدث اتبع مرتين، يتنفذ مرة واحدة.
- **Retry** بـ `backoff`، وبعد آخر محاولة يروح **Dead letter** (`failed_jobs` + تنبيه Sentry) ويتعاد يدوي من Horizon.

شكل الـ Event

```
final readonly class SaleCompleted implements DomainEvent
{
    public const NAME = 'sales.sale_completed';
    public const VERSION = 1;

    public function __construct(
        public string $saleId,
        public string $tenantId,
        public string $branchId,
        public ?string $customerId,
        public int $totalPiasters,
        public int $costPiasters,
        public bool $isCredit,
        /** @var list<array{variant_id: string, qty: int, lot_id: string}> */
        public array $items,
        public CarbonImmutable $occurredAt,
    ) {}
}
```

- الأسماء بصيغة الماضي {module}.{something_happened}، ومعناها **Version** عشان نغيز الشكل من غير ما نكسر الـ Listeners القديمة.
 - الـ Payload فيه اللي الـ Listeners محتاجه (IDs + أرقام)، مش Models كاملة.
 - كل Event بيحمل `tenant_id`، والـ Listener بيرجع الـ Tenant context قبل ما يشتغل.
- Base listener** بيتأكد إن الـ Module بتاعه متفعل للمحل ده، ولو لأ بيتجاهل الحدث:

```
abstract class ModuleListener implements ShouldQueue
{
    abstract protected function module(): string;

    public function handle(DomainEvent $event): void
    {
        if (! Modules::enabled($this->module(), $event->tenantId)) return;
        $this->once($event, fn () => $this->react($event)); // processed_events
    }
}
```

كتالوج الأحداث (البداية)

الحدث	بيطلع من	مين يسمع (Async)
<code>sales.sale_completed</code>	Sales	Reports (ملخص يومي)، Cash (إجمالي الوردية)، Inventory، Realtime (فحص النواقص ← <code>inventory.stock_low</code> ، Messaging (اقتراح رسالة شكر)
<code>sales.sale_returned</code>	Sales	SupplierReturns (لو معيب ← سلة المرتجعات)، Realtime، Reports
<code>sales.sale_voided</code>	Sales	Reports، Audit، Realtime (للمالك)
<code>inventory.stock_low</code>	Inventory	Realtime، Notifications، Suppliers (اقتراح طلبية)

الحدث	يطلع من	مين يسمع (Async)
<code>inventory.item_marked_defective</code>	Inventory	SupplierReturns
<code>purchases.purchase_received</code>	Suppliers	Reports, Inventory (تنبيه زيادة التكلفة)
<code>repairs.ticket_created / repairs.status_changed</code>	Repairs	Realtime (الفنيين), Messaging (رسالة مقترحة), Reports
<code>repairs.ticket_delivered</code>	Repairs	Reports, Cash, Customers (لو آجل — Contract جوه العملية نفسها)
<code>imports.shipment_received</code>	Imports	Reports, Realtime (للمالك)
<code>imports.payment_recorded</code>	Imports	Reports, Audit
<code>supplier_returns.return_settled</code>	Supplier Returns	Reports
<code>cash.shift_closed</code>	Cash	Reports, Realtime (العجز/الزيادة للمالك)
<code>customers.debt_overdue</code> (Scheduler)	Customers	Messaging (تذكير مقترح), Notifications
<code>approvals.requested / approvals.decided</code>	Sales/Identity	Realtime (تطبيق المالك)
<code>modules.module_enabled / module_disabled</code>	Module Manager	Seeders, Cache, Billing (الاستخدام)
<code>billing.subscription_changed</code>	Billing	ModuleManager (تحديث الـ Entitlements), Notifications

- الـ **Real time** (قسم 8) = Listener واحد يحوّل Events مختارة لـ Broadcast على Reverb.
- الـ **Audit log** = Listener ييسجل الأحداث الحساسة.
- الـ **Reports** بتبني **Read models** (زي `daily_sales_summary` , `repair_stats`) من الأحداث بدل ما التقارير تعمل Queries ثقيلة على الجداول الأساسية.
- **مش Event Sourcing**: الجداول هي مصدر الحقيقة، والـ Events للتواصل والتفاعل. (جدول `stock_movements` هو الاستثناء الطبيعي: Ledger ثابت).
- **مستقبلاً**: لو Module انفصل كخدمة لوحده، الـ Relay بيعت الأحداث لـ Broker (RabbitMQ / Redis Streams) بدل الـ Queue الداخلي، من غير تغيير في الـ Modules.

4. Multi-tenancy

قاعدة واحدة + tenant_id في كل جدول بيانات.

```
// app/Support/Tenancy/BelongsToTenant.php
trait BelongsToTenant
{
    protected static function bootBelongsToTenant(): void
    {
        static::addGlobalScope(new TenantScope);

        static::creating(function (Model $model): void {
            $model->tenant_id ??= app(CurrentTenant::class)->id();
        });
    }

    public function tenant(): BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }
}
```

- CurrentTenant يحدد من المستخدم المسجل (ResolveTenant Middleware)، والفرع من X-Branch-Id Header ومتأكد من أن المستخدم له صلاحية عليه).
- **خط دفاع ثاني: Row Level Security** Postgres على الجداول الحساسة (SET app.tenant_id في أول كل request). لو حد نسي ال scope، الداتايز نفسها ترفض.
- **اختبار إجباري** (Pest Arch + Feature): كل Model في Modules/*/Models لازم يستخدم BelongsToTenant (ماعد الاستثناءات المعروفة)، وتست إن مستخدم محل A مش بيشف أي بيانات لمحل B.
- ال Queued Jobs بتشيل tenant_id ويتعمل restore لل context قبل ما تشتغل.
- spatie/laravel-permission ب teams = true و team_id = tenant_id .

5. تصميم قاعدة البيانات

5.1 قواعد عامة

- ال Primary key: uuid (UUIDv7) لكل الجداول الي ممكن تتعمل أوفلاين (...sales, repairs, customers, cash)، و bigint للجداول الداخلية (lookup).
- كل جدول: tenant_id ، created_at ، updated_at و deleted_at (Soft delete) للكيانات الأساسية.
- الفلوس: bigint بالقرش — عمود اسمه *_piasters أو Cast اسمه MoneyCast .
- الكميات: integer (القطع). مفيش كسور في المخزون.

- Index على (tenant_id, ...) في كل حاجة بيتعمل عليها بحث.
- البحث: pg_trgm + Index على اسم الصنف والكود والباركود (بحث عربي/إنجليزي جزئي سريع).

5.2 الجداول الأساسية (المرحلة الأولى)

```

tenants          id, name, slug, phone, tax_id, settings(jsonb), created_at
branches         id, tenant_id, name, address, phone, invoice_prefix, is_active
users            id, tenant_id, name, phone, email, password, pin_hash, is_owner
devices          id(uuid), tenant_id, branch_id, name, platform, last_seen_at,
device_code
categories       id, tenant_id, parent_id, name, type(enum:
accessory|part|device|service)
brands           id, tenant_id, name
device_models    id, tenant_id, brand_id, name, release_year      -- كتالوج مشترك + خاص
products        id, tenant_id, category_id, brand_id, name, sku, track_serial(bool),
is_active
product_variants id, tenant_id, product_id, name, barcode, quality_grade(enum),
price_retail, price_wholesale, price_technician, min_stock
product_compatibility variant_id, device_model_id      -- مركب
stock_levels     tenant_id, branch_id, variant_id, qty, avg_cost      -- PK (branch_id,
variant_id)
stock_lots       id, tenant_id, branch_id, variant_id, source_type(enum:
supplier|import_shipment|opening|customer_return),
source_id, source_ref_id, unit_cost, qty_in, qty_remaining, received_at
stock_movements id, tenant_id, branch_id, variant_id, lot_id, type(enum), qty(+/-),
unit_cost, ref_type, ref_id, user_id, created_at      -- Append-only
serial_items     id, tenant_id, variant_id, branch_id, lot_id, serial, status(enum)
customers        id(uuid), tenant_id, name, phone, phone_e164, notes, credit_limit,
balance
customer_ledger  id, tenant_id, customer_id, type, amount, ref_type, ref_id, created_at
suppliers        id, tenant_id, name, phone, type(enum: local|shop), balance
supplier_ledger  id, tenant_id, supplier_id, type, amount, ref_type, ref_id
purchases / purchase_items
sales            id(uuid), tenant_id, branch_id, device_id, shift_id, number,
customer_id,
status, subtotal, discount, total, paid, due, cost_total, completed_at
sale_items       id, sale_id, variant_id, lot_id, serial_item_id, qty, unit_price,
discount, unit_cost
sale_payments    id, sale_id, method(enum), amount, reference
shifts           id(uuid), tenant_id, branch_id, user_id, opened_at, closed_at,
opening_cash, expected(jsonb), actual(jsonb)
cash_movements / expenses / expense_categories
activity_log      (spatie)

```

5.3 جداول الصيانة

```

fault_categories id, tenant_id, name, sort      -- شاشة، بوردة، سو كيت
...
fault_types      id, tenant_id, category_id, name, default_labor_price, is_active,
sort
intake_checklist_items id, tenant_id, label, sort      -- ... بيفتح؟ بيشحن؟
repair_tickets   id(uuid), tenant_id, branch_id, number, customer_id,
device_model_id, imei, color,
passcode_encrypted, accessories(jsonb), condition(jsonb),
intake_checks(jsonb),

```

	status(enum), technician_id, received_at, received_by, expected_at, delivered_at, labor_price, deposit, total, warranty_days, notes
repair_ticket_faults	ticket_id, fault_type_id, kind(enum: reported diagnosed), note
repair_status_logs	id, ticket_id, from_status, to_status, user_id, note, created_at
repair_parts	id, ticket_id, variant_id, lot_id, qty, unit_price, unit_cost

الأعطال الافتراضية (الجدول الي في PLAN.md) بتعمل **Seed** لكل محل جديد وقت التسجيل.

5.4 جداول الاستيراد ومرتجعات الموردين

import_contacts	id, tenant_id, type(enum: factory agent shipping clearance), name, country, phone, wechat, notes
import_shipments	id, tenant_id, contact_id, number, status(enum), ordered_at, expected_at, arrived_at, cost_allocation(enum: value qty weight), notes
import_shipment_items	id, shipment_id, variant_id, qty_ordered, qty_received, qty_damaged, unit_price, landed_unit_cost, weight
import_costs	id, shipment_id, type(enum: shipping customs clearance transport commission other), contact_id, amount
import_payments	id, tenant_id, contact_id, shipment_id(nullable), amount, method, paid_at,
import_claims	reference, receiver, original_amount_note, proof(media) id, shipment_id, type(enum: shortage damaged), amount, status
supplier_returns	id, tenant_id, source_type, source_id, number, status(enum: pending sent accepted rejected), sent_at, settled_at
supplier_return_items	id, return_id, variant_id, lot_id, serial_item_id, qty, unit_cost, reason(enum), outcome(enum: credit replace refund rejected)

- حساب كل جهة استيراد = Σ (تكلفة الشحنات + المصاريف المنسوبة ليها) - Σ الدفعات - Σ المطالبات المقبولة. بيتحسب في **View** أو Query class, مش عمود بيتعدل بإيد.

5.5 جداول الرسائل والـ Billing

message_templates	id, tenant_id, key, context(enum: repair sale customer supplier import general), name, body, is_active
message_logs	id, tenant_id, template_id, user_id, recipient_phone, context_type, context_id, rendered_body, opened_at
public_links	id(token), tenant_id, linkable_type, linkable_id, expires_at -- لينكات الفواتير والإيصالات
plans, plan_features, subscriptions, subscription_items, billing_invoices, billing_invoice_lines, billing_payments, payment_methods, coupons, credit_ledger, usage_counters, billing_events	

(التفاصيل في PLAN.md قسم 3.9 — الجداول دي **مش** عليها tenant_id scope الأوتوماتيكي لأنها بتاعة المنصة).

5.6 جداول الـ Modules والـ Events

modules	key(PK), name, tier(enum: platform core optional), depends_on(jsonb), is_public, sort
plan_modules	plan_id, module_key
Modules	الباقه فيها أنهى --
tenant_modules	tenant_id, module_key, entitled(bool), state(enum:

```

trial|enabled|disabled|read_only),
        source(enum: plan|addon|trial|manual), trial_ends_at, enabled_at,
disabled_at, settings(jsonb)
domain_events      id(uuid), tenant_id, name, version, aggregate_type, aggregate_id,
payload(jsonb),
        occurred_at, published_at, attempts                                -- ال Outbox
(Index على published_at IS NULL)
processed_events    event_id, listener, processed_at                        -- PK مركّب
(Idempotency)

```

- domain_events يتعمله Partition شهري أو أرشفة بعد 90 يوم عشان حجمه.

6. محرك المخزون (أهم جزء في ال Business logic)

القاعدة: stock_movements هو مصدر الحقيقة (Append-only, مفيش update ولا delete). stock_levels مجرد أرصدة بتتحدث في نفس ال Transaction.

```

final class CompleteSaleAction
{
    public function __construct(
        private readonly StockLedger $stock,           // Inventory\Contracts - لازم ينجح
        مع البيعة
        private readonly CustomerAccounts $accounts,   // Customers\Contracts - لازم ينجح
        مع البيعة
        private readonly EventRecorder $events,        // Support\Events - بيكتب في ال
        Outbox
    ) {}

    public function handle(SaleData $data, User $cashier): Sale
    {
        return DB::transaction(function () use ($data, $cashier): Sale {
            // Idempotency: اتسجل قبل كده (إعادة مزامنة) رجّعه زي ما هو UUID لو ال
            if ($existing = Sale::find($data->id)) {
                return $existing;
            }

            $sale = Sale::create([...]);

            foreach ($data->items as $item) {
                // lockForUpdate على stock_levels + ال Lots ب FIFO استهلاك
                $consumed = $this->stock->consume(
                    branchId: $data->branchId,
                    variantId: $item->variantId,
                    qty: $item->qty,
                    reference: StockReference::sale($sale->id),
                    serialId: $item->serialItemId,
                );
                $sale->items()->create([...$item->toArray(), 'lot_id' => $consumed->lotId,
                'unit_cost' => $consumed->unitCost]);
            }

            $sale->payments()->createMany($data->payments);

            if ($sale->due > 0) {
                $this->accounts->charge($sale->customer_id, $sale->due, reference: "sale:

```

```

{$sale->id}");
    }

    // بستمع للحدث ده Listeners (نواقص، رسائل، Real time، تقارير) الباقي كله
    $this->events->record(SaleCompleted::fromSale($sale));

    return $sale;
  });
}
}

```

- **التكلفة:** متوسط مرجّح (avg_cost) للتقارير، وكل بيعة بتأخذ unit_cost من ال Lot المستهلك (FIFO) — ده اللي بيخّلي المرتجع يعرف مصدره والمكسب دقيق.
- البيع بالسالب مسموح (خصوصاً في الأوفلاين) بس بيطلع تنبيه تسوية.
- **الاستيراد:** عند استلام الشحنة ← حساب landed_unit_cost لكل صنف ← إنشاء stock_lots بمصدر import_shipment
- **المرتجع من عميل:** يرجع لـ Lot بتاعه (سليم) أو يروح supplier_returns بمصدر ال Lot (معيّب).
- **التحويل بين الفروع:** Movement خروج من فرع + دخول لفرع بنفس ال Lot source (عشان المصدر مضيعش).

7. الكاشير الأوفلاين والمزامنة

7.1 على الجهاز (Nuxt PWA)

- Service Worker : @vite-pwa/nuxt يكاش ال App shell فيفتح من غير نت.
- **Dexie (IndexedDB)** فيه نسخة من: الأصناف والأسعار والباركود، المخزون الحالي للفرع، العملاء، الإعدادات، القوالب.
- **Outbox:** كل عملية (بيعة، مرتجع، عميل جديد، حركة خزنة، تذكرة) بتسجل محلياً بـ UUIDv7 وتتحط في طابور.
- ترقيم الفواتير محلياً: {branch_prefix}-{device_code}-{seq} — مفيش تعارض بين الأجهزة.
- حالة الاشتراك متكاشة على الجهاز مع مهلة 7 أيام أوفلاين.

7.2 API

```

POST /api/v1/sync/push { device_id, operations: [ {id, type, payload, created_at}, ... ] }
← { results: [ {id, status: applied|duplicate|rejected, error?} ] }

GET /api/v1/sync/pull?since=<cursor>&entities=products,stock,customers,templates
← { changes: {...}, next_cursor }

```

- ال Push بيتنقّذ بالترتيب وكل عملية Idempotent على ال id.
- ال Pull بـ cursor على updated_at + id (Delta sync)، وأول مرة Full snapshot.

- التعارضات: السيرفر هو المرجع للأسعار والأصناف، والجهاز هو المرجع لعملياته (البيعة التي حصلت حصلت).
- فشل عملية (مثلاً صنف اتمسح) ← تتعلم rejected وتظهر للمدير يحلها يدوي — **عمرها ما تتمسح بصمت.**

7.3 الترتيب في التنفيذ

1. **MVP:** الكاشير أونلاين بس ييكتب في Dexie الأول (Local-first UI) ويبيعت فوراً.

2. **Beta:** تفعيل الأوفلاين الكامل + الـ Outbox + شاشة "عمليات مستتية مزامنة".

8. الـ Real time (Reverb)

مين يسمع	Events	Channel
لوحة الفرع، الكاشير	SaleCompleted , StockLow , RepairStatusChanged , ShiftClosed	private-tenant. {t}.branch.{b}
تطبيق المالك، لوحة المالك	ApprovalRequested (خصم كبير/مرتجع)، DailySummaryReady ملخص المبيعات	private-tenant. {t}.owner
شاشة الفنيين	RepairTicketCreated , RepairStatusChanged	private-tenant. {t}.repairs
المستخدم	إشعارات شخصية	private-user.{u}

- Authorization للـ Channels في routes/channels.php يتأكد من الـ tenant والفرع.
- Flutter . pusher-js + laravel-echo :Nuxt: dart_pusher_channels (بروتوكول Pusher الي Reverb بيدعمه).
- الـ Broadcasting نفسه **Listener** على الـ Domain Events (بعد ما تنشر من الـ Outbox)، فمحدث يستلم حدث لعملية اترجعت، والـ Modules مش عارفة حاجة عن Reverb.
- الـ Channels الخاصة بـ Module مقفول ما بتفتحش (الـ Authorization يتأكد من Modules::enabled).
- Payload صغير (IDs + أرقام أساسية)، والعمل يعمل refetch لو محتاج تفاصيل.

9. الطباعة والباركود

- **المرحلة 1:** طباعة من المتصفح بـ CSS مخصص لـ 80mm/58mm (@page { size: 80mm auto }) — بتشتغل مع أي طابعة حرارية متعرفة على الويندوز.
- **المرحلة 2: QZ Tray** (أو Print agent صغير) لطباعة ESC/POS مباشرة من غير نافذة الطباعة + فتح درج الكاش.
- لبيلات الباركود: JsBarcode + قالب طباعة لمقاسات الليبل الشائعة (مثلاً 25mm×38).

- قراءة الباركود: القارئ USB يشتغل ك Listener ← Keyboard في الـ POS يفرّق بين الكتابة السريعة (سكانر) والكتابة العادية. + مسح بالكاميرا في الموبايل (@zxing/browser).
- الفاتورة A4 و PDF ← PDF: Blade template أو Browsershot لدعم العربي الأحسن).

10. قوالب WhatsApp

```
final class RenderMessageAction
{
    public function handle(MessageTemplate $template, Model $context, User $user):
    RenderedMessage
    {
        $vars = $this->variablesFor($context); // {رقم_التذكرة}, {اسم_العميل},
        {المبلغ}...
        $body = strtr($template->body, $vars);
        $phone = phone($this->recipientPhone($context), 'EG')->formatE164(); // +2010...

        $log = MessageLog::create([...]); // سجل "اتفتحت"
        return new RenderedMessage(
            body: $body,
            url: 'https://wa.me/'.ltrim($phone, '+').'?text='.rawurlencode($body),
        );
    }
}
```

- الـ Frontend يعرض الرسالة للتعديل، وبعد ذلك window.open(url).
- لينك الفاتورة/الإيصال: {token} https://app.domain/r/{token} (جدول public_links , token عشوائي 32 حرف، ومفيهوش بيانات حساسة).
- القوالب الافتراضية بتعمل Seed لكل محل جديد.

11. الـ Billing (تقنياً)

```
interface PaymentGateway
{
    public function createCheckout(BillingInvoice $invoice, PaymentMethodType $method):
    CheckoutSession;
    public function chargeSavedMethod(BillingInvoice $invoice, PaymentMethod $method):
    ChargeResult;
    public function verifyWebhook(Request $request): WebhookEvent; // HMAC
}
// Drivers: PaymobGateway, FawryGateway, ManualGateway (InstaPay/مندوب)
```

- Webhooks: {gateway} POST /webhooks/ ← تحقق HMAC ← billing_events بـ Unique على gateway_ref (Idempotent) ← Job يطبق الدفع.

- **Scheduler يومي:** `billing:generate-invoices` , `billing:charge-due` , `billing:transition-states` (restricted) `past_due` ← `suspended` , `billing:send-reminders` .
- الاشتراك = باقة (`plan_modules`) + `Modules` إضافية (`subscription_items` نوعها `module`). أي تغيير ← `billing.subscription_changed` ← `ModuleManager` يحدّث `tenant_modules.entitled` .
- تجربة `Module` لوحده (مثلاً 14 يوم استيراد) من صفحة الـ `Modules` , وبعدها يا يشترك يا يبقى `read_only` .
- `Middleware` `module:{key}` للـ `Modules` , `Pennantg` للـ `features` الصغيرة جوه الـ `Module` , و `PlanLimits` للحدود (مستخدمين, فروع, أصناف).
- وضع `RESTRICTED`: القراءة والبيع مسموحين, الإنشاء في الكيانات المحددة ممنوع (403 برسالة واضحة + بانر).
- لوحة `Filament`: المحلات, الاشتراكات, الموافقة على تحويلات `InstaPay` (صورة الإيصال), الكوبونات, المندوبين.

12. الـ API Conventions

- `Prefix`: `/api/v1/...` — `Route::apiResource` للـ `CRUD` .
- الرد دائماً `API Resource`: `json { "data": {...}, "meta": {...} }`
- شكل الأخطاء موحّد (Exception handler): `{ "code": "insufficient_credit", "errors": { "field": [...] } }` `json { "message": "الرصيد غير كافي", "code": "insufficient_credit", "errors": { "field": [...] } }`
- الأكواد: 201 إنشاء, 403 422 `Validation` صلاحيات/باقة, 404, 409 تعارض, 429 `Rate limit` .
- `Pagination: Cursor` للقوائم الكبيرة (المبيعات, الحركات), و `Page` للإعدادات.
- فلتر وترتيب: `?filter[status]=ready&sort=-received_at&include=customer` (spatie/query-builder).
- `Rate limiting` على `Login` و `PIN` و `Sync` و `Webhooks` .
- `Accept-Language: ar` افتراضي — رسائل الـ `Validation` بالعربي.

13. الـ Frontend (Nuxt 4)

```
pnpm dlx nuxi@latest init apps/web
pnpm add @nuxt/ui @pinia/nuxt @nuxtjs/i18n @vite-pwa/nuxt @vueuse/nuxt dexie \
  laravel-echo pusher-js jsbarcode @zxing/browser echarts vue-echarts zod
pnpm add -D vitest @vue/test-utils @playwright/test openapi-typescript
```

```
apps/web/app/
├── pages/
│   ├── pos/                # اختصارات (Fullscreen, مستقل layout) الكاشير
│   ├── repairs/            # التذاكر + استلام جهاز
│   └── inventory/ products/ purchases/ suppliers/ customers/
```

```

|   |— imports/ supplier-returns/
|   |— cash/ reports/ settings/ billing/
|— layouts/ default.vue (Sidebar RTL) · pos.vue
|— composables/ useApi.ts · useAuth.ts · useBranch.ts · useModules.ts · useScanner.ts ·
useWhatsApp.ts · useEcho.ts
|— middleware/ module.ts (Module ← "صفحة" فعل ال Module ل Route يمنع دخول)
|— modules-registry.ts # key ← routes + ال + widgets عناصر القائمة
|— stores/ (Pinia) auth · cart · settings · sync
|— offline/ db.ts (Dexie schema) · outbox.ts · sync.ts
|— utils/ money.ts (قرش ≠ جنيه) · phone.ts · print.ts

```

- **RTL أولًا:** dir="rtl" + خط **Cairo** أو IBM Plex Sans Arabic، والأرقام إنجليزي (أوضح في الكاشير) مع إعدادات لتحويلها.
- Types من ال API: openapi-typescript على ال spec اللي Scramble بيطلع ← packages/api-types.
- Auth: Sanctum SPA cookies، وال POS بـ Device token + دخول الكاشير بـ PIN.
- أداء الكاشير: البحث في Dexie (مش في السيرفر)، هدف إضافة صنف > 100ms.
- **ال Modules في الواجهة:** القائمة وال Dashboard widgets والأزرار (زي "تحويل لتذكرة صيانة" في الكاشير) بتبني من modules اللي راجعة من ./me. كل صفحة Module عليها {{ module: 'imports' }}. ويفضل Layers في Nuxt لكل Module كبير (layers/imports, layers/repairs) عشان الكود يفضل منفصل.
- صفحة **"ال Modules"** في الإعدادات: كل ال Modules بوصف وسعر، والمتفعل منها، وزار تجربة/اشترك/إخفاء.

14. تطبيق المالك (Flutter)

- firebase_messaging, dart_pusher_channels, json_serializable + freezed, go_router, dio, flutter_riverpod (Push), mobile_scanner, fl_chart (الجرد بالكاميرا).
- الشاشات: الملخص اللحظي، الموافقات، التذاكر، النواقص، الآجل، الاستيراد (الأرصدة والشحنات)، التقارير.
- Repository + (features/dashboard, features/approvals ...) Architecture: Feature-first لكل Feature.
- **يتبني بعد ما ال API يستقر** (المرحلة P1) — مش من أول يوم.

15. الأمان

- تشفير على مستوى الحقل (cast encrypted): الباسورد/النمط، الرقم القومي. صور البطايق في Bucket خاص بـ Signed URLs قصيرة العمر.
- 2FA للمالك (Fortify). PIN للكاشير مع قفل بعد 5 محاولات.
- إدارة الأجهزة: المالك يشوف الأجهزة المتصلة ويلغي أي Token.
- Audit Log لكل العمليات الحساسة (إلغاء، مرتجع، تعديل سعر، خصم فوق الحد، فتح الدرج، رؤية بيانات مشفرة).

- SameSite=Lax / HttpOnly / Secure Cookies فقط, CSP headers, HTTPS

- composer audit g pnpm audit في ال CI.

16. الاختبارات و ال CI

الطبقة	الأداة	يغطي إيه
Arch	Pest Arch	Controllors مفيهاش DB, كل Model عليه tenant trait, مفيش () env , مفيش Module يستخدم Models Module ثاني (Events g Contracts بس)
Modules	Pest	Route ل Module مقفول ← 403, Listener مايبشغلش لمحل ال Module مقفول عنده, ال Dependencies, الإلغاء مايمسحش داتا
Events	Pest	كل Action بيسجل الحدث الصح في ال assertRecorded : Events (Outbox), ال Listeners Idempotent (نفس الحدث مرتين ← أثر واحد), Rollback ← مفيش حدث
Unit	Pest	Money, توزيع تكلفة الاستيراد, Proration, FIFO, تجهيز رقم الموبايل
Feature	Pest	كل Endpoint: نجاح + Validation + صلاحيات + عزل المحلات
Concurre ncy	Pest	بيعتين في نفس اللحظة لنفس الصنف ← المخزون صح
Frontend unit	Vitest	Cart, الحسابات, ال Outbox
E2E	Playwright	رحلة: فتح وردية ← بيع ← مرتجع ← قفل وردية. واستلام جهاز صيانة ← تسليم. ووضع الأوفلاين (قطع النت وتكملة البيع)

GitHub Actions (على كل PR):

```
api: composer install → pint --test → larastan (level 8) → pest --parallel (Postgres service)
web: pnpm install → nuxi typecheck → eslint → vitest → nuxi build
e2e: (على main) docker compose up → playwright
```

17. النشر (DevOps)

البيئة	المكان
Local	Docker compose / Sail
Staging	VPS صغير (نفس إعداد ال Production) — كل merge على main
Production	VPS (Hetzner/DigitalOcean) 4 vCPU / 8GB في البداية

- **Docker images**: api (FrankenPHP أو php-fpm + nginx), worker (Horizon), reverb, scheduler, web (Nuxt) — والإدارة بـ **Coolify** أو **Laravel Forge**.
- Postgres: على VPS منفصل أو **Backup**, Managed **يومي** (pgBackRest أو pg_dump) لـ Cloudflare R2 + اختبار استرجاع شهري.
- Reverb ورا Nginx على `wss://ws.domain`.
- الملفات على **Cloudflare R2**, والـ DNS و الـ CDN على Cloudflare.
- مراقبة: Telescope, Uptime monitor, Horizon dashboard, (Laravel + Nuxt + Flutter) **Sentry** على الـ Staging بس.
- Migrations بـ zero-downtime: إضافة عمود ← نشر ← Backfill ← إزالة القديم في إصدار بعده.

18. خطة التنفيذ التفصيلية

الأسبوع 0 — التجويز (أول يوم ويومين)

```
# 1) الـ Repo
mkdir -p apps packages infra docs

# 2) Laravel
composer create-project laravel/laravel apps/api
cd apps/api
php artisan install:api          # Sanctum + routes/api.php
php artisan install:broadcasting # Reverb + Echo config
composer require laravel/horizon spatie/laravel-permission spatie/laravel-query-builder \
    spatie/laravel-activitylog brick/money propagandistas/laravel-phone dedoc/scramble
laravel/pennant
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan
laravel/pint
php artisan horizon:install
./vendor/bin/pest --init

# 3) Nuxt
cd ../ && pnpm dlx nuxi@latest init web && cd web
pnpm add @nuxt/ui @pinia/nuxt @nuxtjs/i18n @vite-pwa/nuxt @vueuse/nuxt dexe laravel-echo
pusher-js

# 4) Docker compose (pgsql, redis, mailpit, minio) + CI workflow + .editorconfig
```

- **Postgres** بدل SQLite في `.env`, `APP_LOCALE=ar`, `APP_TIMEZONE=Africa/Cairo`. ✓
- **Pest + Larastan + Pint** شغالين في الـ CI على أول PR. ✓

الأسبوع 1-3 — الأساس (Identity + Catalog)

الأسبوع	المهام
1	Tenancy (scope + middleware + trait + تست العزل)، هيكل الـ Modules (Manifest + ServiceProvider loader + ModuleGate + middleware) ، الـ Outbox (EventRecorder + Relay + ModuleListener + processed_events) ، تسجيل محل جديد باختيار نوع المحل، Login، الفروع، Money cast
2	المستخدمين والأدوار والصلاحيات، الأجهزة و Layout، Audit log، PIN، Auth، Nuxt RTL + اختيار الفرع
3	التصنيفات، الماركات، الموديلات، الأصناف والمتغيرات، التوافق، الباركود، بحث pg_trgm، استيراد أصناف من Excel

الأسبوع 4-8 — MVP البيع

الأسبوع	المهام
4	محرك المخزون (movements, levels, lots, FIFO, avg cost) + تستات الـ Concurrency، الرصيد الافتتاحي
5	المشتريات والموردين وحساباتهم، مرتجع المشتريات
6	الـ POS: شاشة البيع، البحث من Dexie، السلة، الخصومات، طرق الدفع، الفواتير المعلقة، الطباعة 80mm
7	العملاء والآجل وكشف الحساب، المرتجعات من العملاء، الورديات والخزنة والمصروفات
8	التقارير الأساسية + لوحة المالك + (SaleCompleted, StockLow) Reverb + قوالب WhatsApp (الأساس)

الأسبوع 9-11 — الصيانة

- قوائم الأعطال + Checklist الاستلام + Seed افتراضي.
- استلام الجهاز، الإيصال بـ QR، الحالات والـ Timeline، القطع المسحوبة، العمولة، الضمان.
- قوالب رسائل الصيانة + شاشة الفنيين Real time + تقارير الأعطال.

الأسبوع 12-14 — الاستيراد ومرتجعات الموردين

- جهات الاستيراد، الشحنات، المصاريف وتوزيع التكلفة، الاستلام والنواقص.
- الدفعات وكشف الحساب.
- سلة المرتجعات، الفرز حسب المصدر، إذن المرتجع، الحالات والتأثير على الحسابات.

الأسبوع 15-17 — الـ Billing

- الباقات والحدود و Pennant، التجربة، Manual + Fawry + Paymob، الـ Webhooks، الفواتير، الـ Scheduler والتذكيرات، Filament للإدارة.

- الأوفلاين الكامل (Outbox + Sync push/pull)، تحسين السرعة، 10-20 محل تجريبي، إطلاحات، Sentry، Backups، الـ Production.

بعد الإطلاق

- تطبيق المالك (Flutter)، المستعمل، الشحن والتحويلات، الفروع والتحويلات، ETA، QZ Tray، المتجر الأونلاين.

19. Definition of Done (لكل Feature)

- Migration قابلة للرجوع + Indexes.
- Model عليه BelongsToTenant + Policy + Casts.
- الـ Feature جوه Module واحد، والـ Routes عليها module:{key} لو Optional، والصلاحيات في الـ Manifest.
- أي تأثير على Module ثاني عن طريق Contract (لو لازم يبقى في نفس العملية) أو Event (غير كده) — ومتسجل في كتالوج الأحداث.
- Form Request + Action + API Resource.
- تستات Feature (نجاح، Validation، صلاحيات، عزل المحلات).
- الشاشة في Nuxt بالعربي RTL وشغالة على شاشة الموبايل.
- Audit log للعمليات الحساسة.
- Pint + Larastan + Pest + Typecheck كلهم أخضر.
- اتجرب على الـ Staging.

20. أول 5 مهام تبدأ بيها النهارده

1. إنشاء الـ (Postgres/Redis) Docker compose + Nuxt + Laravel + Monorepo.
2. إعداد الـ (Pint + Larastan + Pest) CI من أول commit.
3. Tenant + Branch + User trait الـ Tenancy + تست العزل + هيكل الـ Modules والـ Outbox (قبل أي Feature، عشان كل حاجة بعد كده تتبني عليهم).
4. تسجيل محل جديد (Onboarding API) باختار نوع المحل ← Modules مقترحة، مع Seed للأعطال والقوالب والأدوار.
5. Layout الـ Nuxt بالعربي RTL + Login + اختيار الفرع.